

Lecture 14: Polymorphism

What is Polymorphism?

Polymorphism is an OOP concept that allows one method or object to perform the same action in different ways. It means “many forms”, where a single interface can have multiple implementations. It is achieved through method overloading and method overriding.

Example:

A draw() method can draw circle, rectangle, or triangle.

Types of Polymorphism

1. Compile-Time Polymorphism (Static)

Compile-time polymorphism is the type of polymorphism in which the method call is resolved during compilation (before the program runs). It is achieved through method overloading, where multiple methods have the same name but different parameters.

Happens through Method Overloading.

2. Runtime Polymorphism (Dynamic)

Runtime polymorphism is the process in which the method to be called is decided during program execution (runtime), not during compilation. It happens through method overriding.

Method Overloading

Method overloading is a feature in Java where multiple methods have the same name but different parameters (number or type of arguments) within the same class. It is used to achieve compile-time polymorphism.

Can be done by:

a) Changing number of arguments

```
add(int a, int b)
```

```
add(int a, int b, int c)
```

b) Changing data type

```
add(int a, int b)
```

```
add(double a, double b)
```

Important: Return Type Alone Cannot Overload

Wrong:

```
int add(int a, int b)
```

```
double add(int a, int b)
```

Why?

Because Java gets confused (ambiguity).

Can main() be overloaded?

Yes.

But JVM only runs:

```
public static void main(String[] args)
```

Other overloaded main methods won't run automatically.

Method Overriding

Method overriding is a feature where a child class provides its own implementation of a method that is already defined in the parent class. It is used to achieve runtime polymorphism.

Example:

```
class Animal{  
    void sound(){  
        System.out.println("Animal sound");  
    }  
}
```

```
class Dog extends Animal{  
    void sound(){
```

```
        System.out.println("Bark");  
    }  
}
```

Rules of Overriding

1. Same method name
2. Same parameters
3. Must have inheritance (IS-A relationship)

Can static method be overridden?

No.

Static methods belong to class, not object.

Runtime Polymorphism

Runtime polymorphism is the process in which the method to be called is decided during program execution (runtime), not during compilation. It happens through method overriding.

Example:

```
Bike b = new Splendor();
```

```
b.run();
```

Output:

running safely with 60km

Even though reference is Bike, object is Splendor, so child method runs.

Upcasting

Upcasting is the process where a parent class reference variable refers to a child class object. It is commonly used in runtime polymorphism.

Bike b = new Splendor();

Called upcasting.

Possible Questions (Polymorphism)

Theory Questions

1. Define polymorphism.
2. Explain types of polymorphism.
3. What is method overloading?
4. What is method overriding?
5. Difference between overloading and overriding.

6. Why can't return type alone overload a method?

7. Can main() be overloaded?

8. Can static method be overridden?

9. What is runtime polymorphism?

10. What is upcasting?

Lab Possibilities

1. Make overloaded add() methods.

2. Create parent and child class for overriding.

3. Use upcasting and show runtime polymorphism.

4. Shape example (Circle, Rectangle, Triangle).

Lecture 15: Abstract

What is Abstraction?

Abstraction is the process of hiding implementation details and showing only essential functionality to the user. It helps users focus on what an object does rather than how it works.

Example:

You send SMS but don't know how message travels.

Abstract Class

An abstract class is a class declared with the abstract keyword that cannot be instantiated and may contain both abstract and non-abstract methods. It serves as a blueprint for other classes.

Example:

```
abstract class Bike{  
  
}
```

Features of Abstract Class

Can have abstract methods

Can have normal methods

Cannot create object

Can have constructor

Can have static methods

Can have final methods

Abstract Method

An abstract method is a method declared without a body (implementation). The child class must provide its implementation.

Example:

```
abstract void run();
```

Child Must Implement

```
abstract class Bike{  
    abstract void run();  
}
```

```
class Honda extends Bike{  
    void run(){  
        System.out.println("running safely");  
    }  
}
```

Object Creation

Cannot do:

Bike b = new Bike(); ❌

Can do:

Bike b = new Honda(); ✅

Possible Questions (Abstract)

Theory

1. Define abstraction.
2. What is abstract class?

3. What is abstract method?

4. Can abstract class have constructor?

5. Can abstract class have normal methods?

6. Can we create object of abstract class?

Lab

1. Create abstract class Shape.

2. Add abstract method draw().

3. Create child classes Circle, Rectangle.

4. Call methods using parent reference.